PORTALDOT HACKER HOUSE 2026 · MODULAR CLOUD & MIDDLEWARE

PortalForge

Build, observe, and automate on Portaldot — with evidence you can verify on-chain.

Deploy · Index · Automate · Recover · Verify

github.com/Adrian001-Crypt/portalforge

The problem: Portaldot apps live on two interfaces at once

Two views of one event

Solidity runs inside pallet-revive. Every event is an EVM log **and** a native Revive event. EVM tools miss dispatch errors and runtime versions; Substrate tools miss ABIs.

Fragile automation

Teams glue scripts, cron jobs and webhooks. A lost response or a crashed worker means a job runs twice — or never — and nobody can prove which.

Unclear costs and state

Storage deposits hide inside “gas”, 14-dp tPOT shows as 18 dp, and there is no event filter API. Deployments are hard to plan and to verify later.

Who it is for

User	What they get
App teams on Portaldot	Deploy, register and automate contracts from one console, API and CLI
Contract developers	Live simulation, exact deposit preview, decoded events with raw evidence
Operators	Execution traces, approvals, replay, dead-letter queue, incidents
Node / RPC maintainers	Reachability, latency, block growth and finality from real samples
Ecosystem teams	Verifiable deployment manifests and on-chain execution evidence

One loop, end to end

01

Deploy

Simulate on V3, exact deposit, journaled broadcast, manifest

02

Index

EVM logs + native events, cursors, reorgs, finality

03

Automate

Versioned workflows: condition, HTTP, contract call, evidence

04

Recover

Intent first, query before retry, leases, dead-letter

05

Verify

Console and pf CLI re-read every claim from chain

Delivered as a web console, HTTP API (OpenAPI), TypeScript SDK, pf CLI, three template contracts and a reference receiver.

Architecture

Portaldot V3

eth-rpc (pallet-revive)
Native WS (Revive · Contracts · System)
reviveApi runtime calls

Services

indexer — EVM + native streams
worker — router, scheduler, engine, executor, monitor
API — auth, RBAC, OpenAPI

Interfaces

Web console
TypeScript SDK · pf CLI
Idempotent receivers (demo service)

Shared state: SQLite (WAL)

Every multi-step write is one BEGIN IMMEDIATE transaction: cursor + events together; leases and state changes guarded by owner and expiry.

Least privilege

The API holds no chain key. The worker’s executor only calls allowlisted, nonpayable functions within a fee and rate budget.

Why Portaldot: built on what we measured

Measured on V3 testnet	What PortalForge does with it
EVM block hash = reviveApi.ethBlockHash(n)	Links EVM logs to native Revive events only when this check passes
Storage deposit ≈ 0.06 tPOT per byte; 3.84 tPOT per slot	Exact deposit preview via reviveApi.instantiate; one slot per job
H160 ↔ SS58 mapping rule	Verified against reviveApi; shows which SS58 funds a 0x key
GRANDPA safe/finalized, ~2-block lag	Triggers gated on finality; reorgs orphan events and cancel queued runs
Cancun opcodes, solc 0.8.28 artifacts	Simulated runtime equals the compiled artifact before any signature
No eth_newFilter	Polling indexer with persisted cursors and adaptive ranges

Source: live probe 2026-11-03 · docs/network-capabilities.md · docs/probe/*.json

Proof of function

22
Foundry tests incl. fuzz

66
unit + integration tests, real Anvil chain

7/7
multi-process fault scenarios

8
live V3 smoke checks passed

Plus 8 real-Chrome tests (wallet deploy, lost-hash recovery, two sessions racing), 14/14 webhook delivery scenarios, 11/11 deploy crash-and-budget checks and 17/17 public-visitor checks.

Testnet contract addresses and transactions: pending — deployment is prepared and budgeted, and waits for testnet tokens.

Source: docs/test-results.md · docs/verification-matrix.md · local results use Anvil, not testnet (run 2026-11-04)

Failure is part of the demo

Injected fault	What PortalForge does	Observed result
Response lost after the receiver processed	Outcome “unknown” → query by executionId → reuse stored result	Processed once
Worker SIGKILL mid-request	Lease expires → second worker reconciles the intent	Job completed once
RPC outage (TCP proxy cut)	Cursor holds, incident opens, catch-up after recovery	Incident auto-resolved
Crash between signing and broadcast	Same signed tx re-broadcast — same hash	No duplicate nonce
Full replay of a finished job	Contract dedup + business reconciliation	Refused as conflict

Guarantee stated honestly: no duplicate business effect when the receiver deduplicates by executionId. Not exactly-once against arbitrary services.

Advantages and a business hypothesis

Why teams would pick it

- Portaldot-specific: dual interface, deposits, mapping, finality
- Recovery semantics that are tested, not promised
- Evidence anyone can re-verify with the CLI
- Open source, self-hostable, one database file

Hypothesis (not validated)

- Free: open-source core, self-hosted
- Paid: hosted indexing with long history
- Paid: team workspaces, SSO, alert channels
- Paid: RPC/node observability for operators

No customers or revenue yet. Next step: interviews with Hacker House teams building on V3.

Known limits and roadmap

Not done / not claimed

- ink!/WASM on pallet-contracts
- EVM↔WASM calls, cross-chain features
- Multi-tenant store (single-host SQLite)
- External identity provider; security audit

Next

- Testnet deployment evidence (pending funding)
- ink! adapter once the toolchain is verified on V3
- PostgreSQL store for hosted multi-tenant use

Open source · Apache-2.0 · github.com/Adrian001-Crypt/portalforge
Read-only demo: adrian001-crypt.github.io/portalforge